Group 1 - PRYM

Team Members	
Harleen Kaur	221566252
Katelyn Hopkins	221552211
Jacob Kavanagh	221088380
Sufyan Serdah	221874672

Activity 1: Tester Assignment

1. As a team,
    a. In a document called EECS2311-SectionName-Team#-TakeHomeAssignment.pdf
    b. List the fully implemented and unit/integration tested user stories in your project up to now that have a GUI as well.
2. In the document, explicitly identify who the main developers are of each user story.
3. evenly distribute all stories to team members such that nobody is assigned to a user story that they are heavily involved in its implementation.
4. Write down who is now responsible for testing which stories, and place this document in your team's GitHub.

Big User Story 1: Account & Profile System
As a buyer/seller, I need to be able to create my account and manage my preferences in order to buy/sell cows based on my choices.

Detailed Story	Developer
Buyer account creation	Sufyan
Buyer/Seller login	Sufyan
Buyer profile modification	Shayan
Seller account creation	Sufyan
Seller profile modification	Harleen

General user profile modification	Harleen
Farm listings	Katelyn
Certification management	Sufyan

Big User Story 2: Group Matching System
As a buyer, I want to be able to find and connect with groups that match my preferences.

Detailed Story	Developer
Show groups with matching preferences	Jacob
Join/Leave a group	Jacob
Search for a group	Jacob
View group details	Jacob
Create a group	Jacob
Invite friends to group	Jacob
Claimed cuts per member	Jacob
Creator transfer	Sufyan

Big User Story 3: Farm Search & Listings
As a buyer, I need to be able to search and compare farms to find the best choice for the group.

Detailed Stories	Developer
Show farms	Katelyn
View farm listings	Katelyn
View farm details & ratings	Katelyn
Compare farms	Katelyn

Save farms	Katelyn
Submit a rating	Harleen
Individual cow management	Sufyan
Search bars & filters for farm listings	Harleen
Filter farms by distance	Jacob
Geographic location	Jacob

Big User Story 4: Cow Share Management & Communication
As a buyer, I need to be able to communicate with my group members and with farms to coordinate cow share details.

Detailed Stories	Developer
Group chat with group members	Sufyan
Communication with farms	Sufyan
Seller association	Sufyan
Association state-machine	Sufyan
Real-time notifications	Sufyan

Tester Distribution

S.No.	Detailed Story	Developer	Assigned Tester
1.	Show groups with matching preference	Jacob	Harleen
2.	Join/Leave a group	Jacob	Harleen
3.	View group details	Jacob	Harleen
4.	Search for a group	Jacob	Harleen
5.	Invite friends to group	Jacob	Harleen
6.	Create a group	Jacob	Harleen
7.	Claimed cuts per member	Jacob	Harleen

8.	Creator transfer	Sufyan	Harleen
9.	Buyer Account Creation	Sufyan	Katelyn
10.	Buyer/Seller Login	Sufyan	Katelyn
11.	Buyer Profile Modification	Shayan	Katelyn
12.	General User Profile Modification	Harleen	Katelyn
13.	Seller account creation	Sufyan	Katelyn
14.	Seller association	Sufyan	Katelyn
15.	Certification management	Sufyan	Katelyn
16.	View farm listings	Katelyn	Sufyan
17.	Show farms	Katelyn	Sufyan
18.	View farm details and ratings	Katelyn	Sufyan
19.	Compare farms	Katelyn	Sufyan
20.	Save/Bookmark farms	Katelyn	Sufyan
21.	Geographic location & filter by distance	Jacob	Sufyan
22.	Search bar & filters for farm listings	Harleen	Sufyan
23.	Farm listings	Katelyn	Jacob
24.	Submit a rating	Harleen	Jacob
25.	Group chat with group members	Sufyan	Jacob
26.	Communication with farms	Sufyan	Jacob
27.	Seller profile modification	Harleen	Jacob
28.	Associate state machine	Sufyan	Jacob
29.	Real-time notifications	Sufyan	Jacob
30.	Individual cow management	Sufyan	Jacob

Katelyn - Accounts
Jacob - Farms & communication
Sufyan - Farms
Harleen - Groups

Activity 2: Individual acceptance testing

End-to-end manual test cases

Assigned user stories for testing

1. View farm listings
2. View farm details and ratings
3. Compare farms
4. Save / bookmark farms
5. Geographic location & filter by distance
6. Search bar & filters for farm listings

Test case format

Each test case includes: Preconditions, steps, expected result, DB verification, and passed.

User story 1: View farm listings

TC-VFL-01: Buyer can browse all farm listings after login	
Preconditions	At least 1 seller account exists with a complete profile (shopName, address). A buyer account exists with valid credentials.
Steps	1. Navigate to http://localhost:5173 2. Click “Login” 3. Enter buyer email and password, click Submit 4. Verify redirect to buyer profile page 5. Click “Browse farms” or navigate to /buyer/farmlistings 6. Observe the list of farms displayed
Expected result	All farms registered in the system are visible as cards Each card shows farm name, location, certifications, and rating

	No 4xx/5xx errors in browser console
DB verification	SELECT COUNT(*) FROM seller; — row count should match the number of farm cards shown on screen
Passed?	Yes

TC-VFL-02: Farm listing page loads with no farms (empty state)	
Preconditions	DB has no Seller rows. Buyer is logged in.
Steps	1. Log in as buyer 2. Navigate to http://localhost:5173/buyer/farmlistings 3. Observe the page content
Expected result	Page loads without error An empty state message is shown (e.g., “No farms found”) No spinner stuck indefinitely
DB verification	SELECT COUNT(*) FROM seller; returns 0
Passed?	Yes

TC-VFL-03: Unauthenticated user is redirected away from farm listings	
Preconditions	User is not logged in (no active session cookie)
Steps	1. Open browser in incognito mode 2. Navigate directly to http://localhost:5173/buyer/farmlistings 3. Observe what happens
Expected result	User is redirected to the login page Farm data is not displayed without authentication
DB verification	N/A
Passed?	Yes

TC-VFL-04: Farm listing page shows all farms from multiple sellers	
Preconditions	At least 3 seller accounts exist with different shop names. Buyer is logged in.
Steps	1. Log in as buyer 2. Navigate to http://localhost:5173/buyer/farmlistings

	3. Count the farm cards displayed
Expected result	All 3+ farms appear Each card shows a unique shop name matching a seller in DB Listings are displayed in a consistent order
DB verification	SELECT shop_name FROM seller; — all names should be visible in the UI
Passed?	Yes

User story 2: View farm details and ratings

TC-FDR-01: Buyer views complete farm detail page	
Preconditions	A seller exists with a fully filled profile (name, address, description, at least one certification). Buyer is logged in.
Steps	1. Log in as buyer 2. Navigate to http://localhost:5173/buyer/farmlistings 3. Click on any farm card 4. Observe the detail page that opens at /buyer/farmlistings/:farmname
Expected result	Farm name, address, and description are shown Certification badges are visible (e.g., Kosher, Organic) Ratings section shows average score and total rating count “Save farm” button is visible
DB verification	SELECT shop_name, shop_address, description, average_rating, total_ratings FROM seller WHERE id = {id}; — values match what is shown on screen
Passed?	Yes

Bugs caught by TC-FDR-01:

Bug report: BUG-04 — averageRating and totalRatings had no @Column annotation and were declared twice	
GitHub issue	Issue #93
Area	Rating system / seller model
Description	The Seller model had averageRating and totalRatings fields with no @Column(nullable = false) and no Java defaults. An intermediate commit accidentally introduced duplicate declarations that caused a compile error.
Fix applied	Deduplicated the fields with proper annotations: @Column(nullable = false) private double averageRating = 0.0;

	@Column(nullable = false) private int totalRatings = 0;
Commit	76bd5bd, 1b7b196
Status	Closed (Fixed)

TC-FDR-02: Buyer views farm ratings — farm with multiple ratings	
Preconditions	A seller has at least 3 ratings submitted by different buyers. Buyer is logged in.
Steps	1. Log in as buyer 2. Navigate to http://localhost:5173/buyer/farmlistings and click on a farm that has ratings 3. Scroll to the ratings section
Expected result	Average rating is displayed (e.g., “4.3 / 5”) Total number of ratings is shown (e.g., “3 reviews”) Individual rating entries are listed
DB verification	SELECT AVG(score), COUNT(*) FROM rating WHERE seller_id = {id}; — values match the UI average and count
Passed?	Yes

Bugs caught by TC-FDR-02:

Bug report: BUG-01 — Rating code could be used more than once	
GitHub issue	Issue #90
Area	Rating system
Description	The submitRating flow used a non-atomic read-check-write pattern: findByCode(code), check isUsed(), then setUsed(true) and save. Two concurrent requests could both pass the isUsed() check before either writes used = true, allowing the same rating code to be redeemed twice.
Fix applied	Replaced the read-check-write pattern with an atomic SQL UPDATE via a new @Modifying repository method: @Query("UPDATE RatingCode rc SET rc.used = true WHERE rc.code = :code AND rc.used = false") int markAsUsed(@Param("code") String code); If 0 rows are affected, the code was already used (or never existed) and the service throws "Invalid or already-used code" immediately.
Commit	d5f56fa
Status	Closed (Fixed)

Bug report: BUG-02 — Rating average was recalculated from full table on every submission	
GitHub issue	Issue #91
Area	Rating system
Description	After saving a new rating, the service re-queried all ratings from the database to recompute the average: List<Rating> allRatings = ratingRepository.findBySeller(seller); This is O(n) per submission and can produce stale values under concurrent load. getFarmRatings had the same problem.

Fix applied	Replaced with an incremental running average:
	int newTotal = seller.getTotalRatings() + 1;
	double newAverage = ((seller.getAverageRating() * seller.getTotalRatings()) + score) / newTotal;
	getFarmRatings now reads the cached fields directly instead of recalculating.

Commit	1b7b196, a864563
Status	Closed (Fixed)

Bug report: BUG-03 — Lost-update race condition on seller rating fields	
GitHub issue	Issue #92
Area	Rating system
Description	Even after fixing the average formula, two concurrent rating submissions could both read the same (averageRating, totalRatings) snapshot, compute independently, and one write silently overwrites the other — effectively dropping a rating.
Fix applied	Added a pessimistic write lock to SellerRepository: @Lock(LockModeType.PESSIMISTIC_WRITE) @Query("SELECT s FROM Seller s WHERE s.id = :id") Optional<Seller> findByIdForUpdate(@Param("id") Long id); submitRating now calls findByIdForUpdate() so concurrent submissions serialize at the DB level.
Commit	a864563
Status	Closed (Fixed)

TC-FDR-03: Buyer views farm ratings — brand new farm with zero ratings	
Preconditions	A seller exists with no ratings at all. Buyer is logged in.

Steps	1. Log in as buyer 2. Navigate to http://localhost:5173/buyer/farmlistings and click on the farm with no ratings 3. Scroll to the ratings section
Expected result	Rating section shows “0 reviews” or “No ratings yet” Average displays as 0.0 or “N/A” No error or blank screen
DB verification	SELECT COUNT(*) FROM rating WHERE seller_id = {id}; returns 0
Passed?	Yes

TC-FDR-04: Farm detail page for a non-existent farm returns an error	
Preconditions	Buyer is logged in
Steps	1. Log in as buyer 2. Manually navigate to /buyer/farmlistings/99999 (an ID that does not exist in DB) 3. Observe the page
Expected result	Page shows a “Farm not found” error message or a 404 page Application does not crash
DB verification	SELECT COUNT(*) FROM seller WHERE id = 99999; returns 0
Passed?	Yes

TC-FDR-05: Rating data is accessible publicly without login	
Preconditions	A seller with at least one rating exists. A second browser or incognito window has no active session.
Steps	1. Open an incognito browser window (simulating a visitor with no account) 2. Navigate to http://localhost:5173 3. Observe the landing page — do not log in 4. Try navigating directly to http://localhost:5173/buyer/farmlistings 5. Observe what happens (redirect to login or farm list shown)
Expected result	If farms require login: user is redirected to /login — document this as the observed behaviour If public browsing is supported: farm listings load and ratings are visible without an account Either way, no server crash or blank white screen

DB verification	SELECT score FROM rating WHERE seller_id = (SELECT id FROM seller WHERE user_id = (SELECT id FROM user WHERE username = 'farmuser1'));
Passed?	Yes

Bugs caught by TC-FDR-05:

Bug report: BUG-05 — No security config rules for rating endpoints	
GitHub issue	Issue #94
Area	Security configuration
Description	Spring Security had no rules for /api/ratings/**. The /api/seller/** catch-all was blocking buyers from hitting /api/ratings/submit, and /api/ratings/generate-code was not locked to sellers.
Fix applied	Added explicit security rules: .requestMatchers("/api/ratings/generate-code").hasRole("SELLER") .requestMatchers("/api/ratings/submit").hasRole("BUYER") .requestMatchers(HttpMethod.GET, "/api/ratings/**").permitAll()
Commit	f191cf8
Status	Closed (Fixed)

User story 3: Compare farms (matching farms by group certifications)

TC-CF-01: Group gets perfect-match farms when seller has all required certifications	
Preconditions	A buyer group requires Organic certification. A seller has Organic certification. Buyer is a member of that group.
Steps	1. Log in as buyer 2. Navigate to http://localhost:5173/buyer/browse-groups and click on the group to open /buyer/groups/:groupId 3. Scroll to or click “Find matching farms”
Expected result	The Organic-certified seller appears under “Perfect matches” Distance to seller is displayed (if buyer has a location set)
DB verification	SELECT * FROM certification WHERE seller_id = {id} AND name = 'ORGANIC'; — record exists
Passed?	Yes

Bugs caught by TC-CF-01:

Bug report: BUG-06 — Invalid certification name threw a cryptic internal error	
GitHub issue	Issue #95
Area	Certifications
Description	(Same as reported under TC-SFF-02 — also caught by this test case during group matching with invalid cert names.)
Fix applied	See fix under TC-SFF-02.
Commit	38d3b27
Status	Closed (Fixed)

TC-CF-02: Partial match when seller has only some required certifications	
Preconditions	Buyer group requires Organic + Kosher. A seller only has Organic. Buyer is in that group.
Steps	1. Log in as buyer 2. Navigate to http://localhost:5173/buyer/browse-groups and click on the group to open /buyer/groups/:groupId 3. View the matching farms section
Expected result	The seller appears under “Partial matches” (not perfect matches) The missing certification (Kosher) is indicated on the card
DB verification	SELECT name FROM certification WHERE seller_id = {id}; — only returns ORGANIC, not KOSHER
Passed?	Yes

TC-CF-03: Seller is excluded when they have no required certifications	
Preconditions	Buyer group requires Halal. A seller has zero certifications. Buyer is in that group.
Steps	1. Log in as buyer 2. Navigate to http://localhost:5173/buyer/browse-groups and click on the group to open /buyer/groups/:groupId 3. View the matching farms section
Expected result	The uncertified seller does not appear in either perfect matches or partial matches
DB verification	SELECT COUNT(*) FROM certification WHERE seller_id = {id}; returns 0

Passed?	Yes

TC-CF-04: Group with no certification requirements matches all farms	
Preconditions	A buyer group has no certifications required. Multiple sellers exist with various certifications. Buyer is in that group.
Steps	1. Log in as buyer 2. Navigate to http://localhost:5173/buyer/browse-groups and click on the group to open /buyer/groups/:groupId 3. View matching farms
Expected result	All sellers appear under “Perfect matches” Farms are sorted by distance ascending (nearest first)
DB verification	SELECT COUNT(*) FROM seller; — count should match number of perfect matches shown
Passed?	Yes

User story 4: Save / bookmark farms

TC-SBF-01: Buyer saves a farm and it appears in saved farms list	
Preconditions	Buyer is logged in. At least one farm exists. Buyer has not previously saved this farm.
Steps	1. Log in as buyer 2. Navigate to http://localhost:5173/buyer/farmlistings 3. Click on a farm card to open the detail page 4. Click the “Save farm” button 5. Navigate to http://localhost:5173/buyer/saved_farms
Expected result	Farm now appears in the saved farms list Save button on detail page changes state to “Saved” or “Remove” No duplicate entries in the list
DB verification	SELECT * FROM saved_farms WHERE buyer_id = {buyerId} AND seller_id = {sellerId}; — row exists
Passed?	Yes

TC-SBF-02: Buyer removes a saved farm

Preconditions	Buyer has at least one farm already saved
Steps	1. Log in as buyer 2. Navigate to http://localhost:5173/buyer/saved_farms 3. Click “Remove” on one of the saved farms 4. Observe the updated list
Expected result	Farm is removed from the list immediately Remaining saved farms are still shown Removed farm no longer appears after page refresh
DB verification	SELECT * FROM saved_farms WHERE buyer_id = {buyerId} AND seller_id = {removedSellerId}; — row does not exist
Passed?	Yes

Bugs caught by TC-SBF-02:

Bug report: BUG-07 — Endpoint was renamed on backend but frontend still calling the old URL	
GitHub issue	Issue #96
Area	Saved farms
Description	The remove-saved-farm endpoint was renamed from /api/buyer/all to /api/buyer/saved-farms on the backend. farm.js was never updated, so every unsave attempt returned 404.
Fix applied	Updated farm.js to call the new endpoint URL /api/buyer/saved-farms.
Commit	f6a3112
Status	Closed (Fixed)

TC-SBF-03: saved farms list is empty for a new buyer	
Preconditions	A brand new buyer account exists with no saved farms
Steps	1. Log in as the new buyer 2. Navigate to http://localhost:5173/buyer/saved_farms
Expected result	Page loads successfully Empty state message is shown (e.g., “No saved farms yet”) No error or crash
DB verification	SELECT COUNT(*) FROM saved_farms WHERE buyer_id = {newBuyerId}; returns 0
Passed?	Yes

TC-SBF-04: Saving the same farm twice does not create a duplicate	
Preconditions	Buyer has already saved farm A
Steps	1. Log in as buyer 2. Navigate to http://localhost:5173/buyer/farmlistings 3. Click on farm A (already saved) 4. Click “Save farm” again if the button is still visible 5. Navigate to http://localhost:5173/buyer/saved_farms and check for duplicates
Expected result	Farm appears only once in the saved farms list No duplicate db rows are created No server 500 error (request is silently skipped or deduplicated)
DB verification	SELECT COUNT(*) FROM saved_farms WHERE buyer_id = {buyerId} AND seller_id = {sellerId}; returns exactly 1
Passed?	Yes

User story 5: Geographic location & filter by distance

TC-GLD-01: Farm cards display distance from buyer’s location	
Preconditions	Buyer account has a zipCode set with resolved latitude/longitude. At least one seller has a registered address with coordinates in DB.
Steps	1. Log in as buyer 2. Navigate to http://localhost:5173/buyer/farmlistings 3. Observe each farm card
Expected result	Distance is shown on each farm card (e.g., “5.3 miles away”) Distance is formatted with one decimal place Unit is miles
DB verification	Manually compute Haversine distance between buyer lat/lon and seller lat/lon — result should match displayed value within rounding
Passed?	Yes

TC-GLD-02: Distance shows “< 1 mile” for very nearby farms	
Preconditions	Buyer and a seller share the same or near-identical zip code / coordinates
Steps	1. Log in as a buyer whose zip code is identical to a specific seller’s zip code 2. Navigate to http://localhost:5173/buyer/farmlistings

	3. Observe the distance label on that seller's card
Expected result	Distance displays as "< 1 mile" rather than "0.0 miles"
DB verification	SELECT latitude, longitude FROM user WHERE id = {buyerId}; and SELECT latitude, longitude FROM user WHERE id = {sellerUserId}; — coordinates are identical or within < 1 mile
Passed?	Yes

TC-GLD-03: Farm without coordinates shows missing distance gracefully	
Preconditions	A seller exists in the database with null latitude and longitude
Steps	1. Log in as buyer 2. Navigate to http://localhost:5173/buyer/farmlistings 3. Find the farm card for the seller with no coordinates
Expected result	Distance field is blank, shows "Distance unavailable", or is simply omitted No crash, no NullPointerException visible to the user
DB verification	SELECT latitude, longitude FROM user WHERE id = {sellerUserId}; — both return null
Passed?	Yes

TC-GLD-04: Matching farms in group detail are sorted nearest-first	
Preconditions	Buyer has a location set. At least 3 farms all qualify as perfect matches for the buyer's group. Each farm has a different distance from the buyer.
Steps	1. Log in as buyer 2. Navigate to http://localhost:5173/buyer/browse-groups and click on the group to open /buyer/groups/:groupId 3. View the matching farms list on the group detail page
Expected result	Farms appear in ascending distance order (nearest farm first) Distance is displayed next to each farm name
DB verification	Compare distances computed from SELECT latitude, longitude FROM user WHERE id IN ({sellerUserIds}); — order should be ascending
Passed?	Yes

User story 6: Search bar & filters for farm listings

TC-SFF-01: Buyer searches for a farm by name	
Preconditions	At least 3 farms with distinct names exist. Buyer is logged in.
Steps	1. Log in as buyer 2. Navigate to http://localhost:5173/buyer/farmlistings 3. Type part of a known farm name into the search bar (e.g., “Green”) 4. Observe the filtered results
Expected result	Only farms whose name contains “Green” are shown Farms not matching the query disappear from the list
DB verification	SELECT shop_name FROM seller WHERE LOWER(shop_name) LIKE '%green%'; — returned names should match exactly what is displayed
Passed?	Yes

TC-SFF-02: Buyer filters farms by certification type	
Preconditions	Some farms have Organic certification, some do not. Buyer is logged in.
Steps	1. Log in as buyer 2. Navigate to http://localhost:5173/buyer/farmlistings 3. Select “Organic” from the certification filter dropdown 4. Observe the results
Expected result	Only farms with Organic certification are shown Non-organic farms are hidden from the list
DB verification	SELECT seller_id FROM certification WHERE name = 'ORGANIC'; — IDs should match those shown in UI
Passed?	Yes

Bugs caught by TC-SFF-02:

Bug report: BUG-06 — Invalid certification name threw a cryptic internal error	
GitHub issue	Issue #95
Area	Certifications
Description	Certification.CertificationType.valueOf(name) threw IllegalArgumentException with the full internal class path leaking to the frontend. Gave no hint of what valid values looked like.
Fix applied	Wrapped in try/catch with a readable message listing valid options:

	catch (IllegalArgumentException e) {
	String valid = Arrays.stream(CertificationType.values()).map(Enum::name).collect(Collectors.joining(", "));
	throw new RuntimeException("Invalid certification: " + name + ". Valid options: " + valid);
	}

Commit	38d3b27
Status	Closed (Fixed)

TC-SFF-03: Buyer filters farms by minimum rating	
Preconditions	Farms exist with varying average ratings (e.g., some at 3.0, some at 4.5). Buyer is logged in.
Steps	1. Log in as buyer 2. Navigate to http://localhost:5173/buyer/farmlistings 3. Set the minimum rating filter to 4.0 4. Observe the filtered list
Expected result	Only farms with average_rating >= 4.0 are shown Farms rated below 4.0 are hidden
DB verification	SELECT shop_name, average_rating FROM seller WHERE average_rating >= 4.0; — results should match UI
Passed?	Yes

TC-SFF-04: Buyer combines search bar and certification filter	
Preconditions	Multiple farms exist; some are named “Valley” and some have Organic certification (with overlap). Buyer is logged in.
Steps	1. Log in as buyer 2. Navigate to http://localhost:5173/buyer/farmlistings 3. Type “Valley” in the search bar 4. Also select “Organic” from the certification filter
Expected result	Only farms that both contain “Valley” in the name and have Organic certification are shown Results are the intersection of both filters

DB verification	SELECT s.shop_name FROM seller s JOIN certification c ON c.seller_id = s.id WHERE LOWER(s.shop_name) LIKE '%valley%' AND c.name = 'ORGANIC'; — should match displayed results
Passed?	Yes

TC-SFF-05: Search with no matching results shows empty state	
Preconditions	Buyer is logged in
Steps	1. Log in as buyer 2. Navigate to http://localhost:5173/buyer/farmlistings 3. Type a nonsense string into the search bar (e.g., “zzzzzzz”)
Expected result	Farm list is empty Empty state message is shown (e.g., “No farms match your search”) No error or crash
DB verification	SELECT COUNT(*) FROM seller WHERE LOWER(shop_name) LIKE '%zzzzzzz%'; returns 0
Passed?	Yes

TC-SFF-06: Buyer sorts farms by rating (highest first)	
Preconditions	At least 3 farms exist with different average ratings. Buyer is logged in.
Steps	1. Log in as buyer 2. Navigate to http://localhost:5173/buyer/farmlistings 3. Select “Sort by rating” (descending) from the sort dropdown 4. Observe the order of farm cards
Expected result	Farms are reordered with the highest-rated farm at the top Order matches a descending sort of average_rating
DB verification	SELECT shop_name, average_rating FROM seller ORDER BY average_rating DESC; — order should match UI
Passed?	Yes

Summary table

Test ID	User story	Scenario	Expected outcome
TC-VFL-01	View farm listings	Logged-in buyer browses all farms	All farms shown as cards
TC-VFL-02	View farm listings	No farms in DB	Empty state message
TC-VFL-03	View farm listings	Unauthenticated access	Redirect to login
TC-VFL-04	View farm listings	Multiple sellers exist	All sellers visible
TC-FDR-01	Farm details & ratings	Full farm detail page	Name, address, certs, ratings shown
TC-FDR-02	Farm details & ratings	Farm with multiple ratings	Average and count correct
TC-FDR-03	Farm details & ratings	Farm with zero ratings	Zero state shown gracefully
TC-FDR-04	Farm details & ratings	Non-existent farm ID	404 / error message
TC-FDR-05	Farm details & ratings	Public ratings endpoint	Returns 200 without login
TC-CF-01	Compare farms	Seller has all required certs	Appears as perfect match
TC-CF-02	Compare farms	Seller has partial certs	Appears as partial match
TC-CF-03	Compare farms	Seller has no required certs	Excluded from results
TC-CF-04	Compare farms	Group has no cert requirements	All farms are perfect matches
TC-SBF-01	Save / bookmark farms	Save a farm	Appears in saved farms list
TC-SBF-02	Save / bookmark farms	Remove a saved farm	Removed from list and db
TC-SBF-03	Save / bookmark farms	New buyer with no saves	Empty state shown
TC-SBF-04	Save / bookmark farms	Save the same farm twice	No duplicate created
TC-GLD-01	Geographic location	Buyer and seller have coordinates	Distance shown on cards
TC-GLD-02	Geographic location	Buyer and seller very close	Shows “< 1 mile”
TC-GLD-03	Geographic location	Seller has no coordinates	Graceful missing distance

TC-GLD-04	Geographic location	Multiple matching farms in group	Sorted nearest-first
TC-SFF-01	Search & filters	Search by farm name	Filtered by name
TC-SFF-02	Search & filters	Filter by certification	Only matching certs shown
TC-SFF-03	Search & filters	Filter by minimum rating	Only high-rated farms shown
TC-SFF-04	Search & filters	Combine search + cert filter	Intersection of both filters
TC-SFF-05	Search & filters	Search with no matches	Empty state shown
TC-SFF-06	Search & filters	Sort by rating descending	Highest rated first

Bug-to-test traceability matrix

GitHub bug	Bug doc #	Caught by test(s)	Bug title
BUG-01	BUG-01	TC-FDR-02	Rating code could be used more than once
BUG-02	BUG-02	TC-FDR-02	Rating average recalculated from full table
BUG-03	BUG-03	TC-FDR-02	Lost-update race on seller rating fields
BUG-04	BUG-04	TC-FDR-01, TC-FDR-02	Missing @Column and duplicate field declarations
BUG-05	BUG-05	TC-FDR-05	No security config rules for rating endpoints
BUG-06	BUG-09	TC-SFF-02, TC-CF-01/02/03	Invalid cert name threw cryptic internal error
BUG-07	BUG-13	TC-SBF-02	Frontend calling old endpoint URL after rename

Activity 3: Individual code review

Detected code smells

Smell 1 — Duplicated badge rendering logic	
Category	Duplicated code (DRY violation)
Severity	Medium
File(s)	FarmListingsPage.jsx lines 68–86, SavedFarms.jsx lines 31–49

Description	Both FarmListingsPage.jsx and SavedFarms.jsx contain nearly identical certification badge rendering blocks. Each file independently checks c.name against five string literals and manually constructs a with inline styles. Both files also define nearly identical badge style objects at the bottom of the file. Any new certification type requires changes in two separate files, and any styling update must be made twice. FarmListing.jsx (the detail page) has already solved this correctly using a BADGE_STYLES lookup table and a CERT_LABELS map — the fix exists in the codebase but wasn't applied to the other two pages.
Code example	// FarmListingsPage.jsx lines 70-85 (duplicated in SavedFarms.jsx lines 33-48) {c.name === "KOSHER" && (<span style={{ ...styles.badge, ...styles.badgeKosher }}>Kosher)} {c.name === "HALAL" && (<span style={{ ...styles.badge, ...styles.badgeHalal }}>Halal)} // ... repeated for ORGANIC, GRASS_FED, NON_GMO
Proposed fix	Extract the badge logic into a shared component (CertBadge.jsx) and import BADGE_STYLES / CERT_LABELS from a shared module. In both FarmListingsPage.jsx and SavedFarms.jsx, replace the if-chain with: {(farm.certifications || []).map(c => <CertBadge certName={c.name} />)}

Fix code	// uiserver/src/components/CertBadge.jsx
	const BADGE_STYLES = {
	  KOSHER: { backgroundColor: '#e3f2fd', color: '#1565c0' },
	  HALAL: { backgroundColor: '#fff3e0', color: '#e65100' },
	  // ... etc
	};
	export default function CertBadge({ certName }) {
	  const style = BADGE_STYLES[certName] || { backgroundColor: '#eee', color: '#555' };
	  return <span style={{ ...badgeBase, ...style }}>{CERT_LABELS[certName]};
	}

Smell 2 — Calling Java getter methods on plain JSON objects	
Category	Inappropriate intimacy / bug
Severity	High (runtime error)
File(s)	FarmListingsPage.jsx line 92, SavedFarms.jsx line 55

Description	Both listing pages attempt to display a farm's profile picture using Java-style getter method calls on what is actually a plain JavaScript object deserialized from JSON: farm.getUser().getProfilePicture() JSON deserialization produces plain objects — there are no getUser() or getProfilePicture() methods in JavaScript. Calling .getUser() throws TypeError: farm.getUser is not a function at runtime, which means farm images are completely broken on both listing pages. The correct pattern (farm.user?.profilePicture) is already used in FarmListing.jsx. Additional bug: the original code checks user?.profilePicture (the logged-in buyer) to decide whether to render the farm's image — wrong object.

Code example	// FarmListingsPage.jsx line 92
	<img src={farm.getUser().getProfilePicture()} ...
	// SavedFarms.jsx line 55
	<img src={farm.getUser().getProfilePicture()} ...

Proposed fix	Replace both broken method call chains with the correct JavaScript property access: farm.user?.profilePicture Also fix the condition to check farm.user?.profilePicture (the farm's image) instead of user?.profilePicture (the buyer's image).
Fix code	// After: {farm.user?.profilePicture ? (<img src={farm.user.profilePicture} alt="farm_photo" style={{ width: '100%', height: '100%', borderRadius: '50%', objectFit: 'cover' }} />) : (<span style={{ color: '#aaa', fontSize: '14px' }}>No image)}
Commit	fix: use correct JSON property access for farm images

Smell 3 — React component named with lowercase letter	
Category	Naming convention violation
Severity	Medium
File(s)	SavedFarms.jsx line 8
Description	The React component is defined with a lowercase function name: function savedFarms(){} React requires all component names to start with an uppercase letter to distinguish between HTML native elements (lowercase) and custom components (uppercase).

	If referenced in JSX as <savedFarms />, React would treat it as an unknown HTML element and render nothing. Currently works only because it is invoked via React Router, not directly in JSX — but this is fragile.

Code example	// SavedFarms.jsx line 8
	function savedFarms(){
	  // …
	}
	export default savedFarms;

Proposed fix	Rename the function declaration to SavedFarms and update the export.

Fix code	// After:
	function SavedFarms(){
	  // …
	}
	export default SavedFarms;

Commit	fix: rename savedFarms component to SavedFarms (PascalCase)

Smell 4 — Silent error handling (catch(console.error))	
Category	Improper error handling
Severity	Medium
File(s)	FarmListingsPage.jsx line 29, SavedFarms.jsx line 21, FarmListing.jsx lines 53–56
Description	All three farm-related pages swallow API errors silently with .catch(console.error). When the backend is unavailable or returns an error, the user sees an empty page with no explanation. The error is only visible in the browser developer console, which no regular user will ever check.

Code example	// FarmListingsPage.jsx line 29
	getAllFarms().then(setFarms).catch(console.error);
	// SavedFarms.jsx line 21
	getSavedFarms().then(setFarms).catch(console.error);

Proposed fix	Add an error state variable and display a user-facing message on failure. Replace .catch(console.error) with a handler that sets an error state: .catch((err) => { console.error(err); setError('Could not load farm listings.'); }); In JSX, show the error message when error is set.

Fix code	const [error, setError] = useState(null);
	useEffect(() => {
	  getAllFarms()
	  .then(setFarms)
	  .catch((err) => {
	  console.error(err);
	  setError('Could not load farm listings. Please try again later.');
	  });
	}, []);

Smell 5 — getAllFarms duplicated in ratings.js	
Category	Duplicated code (DRY violation)
Severity	Low-medium
File(s)	farm.js lines 5–8, ratings.js lines 6–9
Description	ratings.js defines its own private copy of getAllFarms() that is byte-for-byte identical to the exported one in farm.js. If the /api/seller/all endpoint URL ever changes, or if fetch options need to be added, it must be updated in two places.

Code example	// ratings.js lines 6-9 (private duplicate)
	async function getAllFarms() {
	  const response = await fetch('/api/seller/all');
	  return response.json();
	}

Proposed fix	Delete the local copy from ratings.js and import from farm.js instead.

Fix code	// ratings.js - add import:
	import { getAllFarms } from './farm';
	// Then delete lines 6-9 (the local duplicate)

Commit	refactor: remove duplicate getAllFarms from ratings.js

Smell 6 — getFarm(id) fetches all farms to find one	
Category	Inefficient algorithm / lazy class
Severity	Medium
File(s)	farm.js lines 12–15
Description	There is no backend endpoint for a single farm by ID, so the frontend fetches the entire farm list on every detail page load just to find one record.

	This makes a network request that returns potentially hundreds of records, sends all of them over the wire, and discards all but one. The FarmListing.jsx detail page makes two full-list fetches on load (one for getFarm, one for getSavedFarms).

Code example	// farm.js lines 12-15
	export async function getFarm(id) {
	  const farms = await getAllFarms();
	  return farms.find((f) => f.id === Number(id));
	}

Proposed fix	Immediate fix: memoize or cache the getAllFarms result in a React context. Correct fix: add a backend endpoint to SellerController.java that returns a single farm by ID, then update farm.js to call it directly.

Fix code	// Backend: SellerController.java
	@GetMapping("/{sellerId}")
	public ResponseEntity<?> getFarmById(@PathVariable Long sellerId) { … }
	
	// Frontend: farm.js
	export async function getFarm(id) {
	  const response = await fetch(`/api/seller/${id}`);
	  if (!response.ok) throw new Error('Farm not found');
	  return response.json();
	}

Smell 7 — Long if-else chain for sorting (switch statement smell)	
Category	Switch statements
Severity	Low-medium
File(s)	FarmListingsPage.jsx lines 54–62
Description	The sorting logic uses a chain of four else-if branches, each containing an inline sort comparator. Adding a new sort option requires adding another else-if branch, violating the open/closed principle.

Code example	if(sortBy === 'name-asc'){
	  filteredFarms = […filteredFarms].sort((a,b) => …);
	} else if (sortBy === 'name-desc'){
	  filteredFarms = […filteredFarms].sort((a,b) => …);
	} else if (sortBy === 'rating-high'){
	  // …
	}

Proposed fix	Replace with a comparator lookup table. Adding a new sort option now only requires adding one entry.
Fix code	const SORT_COMPARATORS = {

	'name-asc': (a, b) => (a.shopName || '').localeCompare(b.shopName || ''),
	'name-desc': (a, b) => (b.shopName || '').localeCompare(a.shopName || ''),
	'rating-high': (a, b) => b.averageRating - a.averageRating,
	'rating-low': (a, b) => a.averageRating - b.averageRating,
	};
	const comparator = SORT_COMPARATORS[sortBy];
	if (comparator) filteredFarms = [...filteredFarms].sort(comparator);

Commit	refactor: replace sort if-else chain with lookup table

Smell 8 — RuntimeException used for all business logic errors	
Category	Inappropriate exception type
Severity	Medium
File(s)	SellerService.java, BuyerService.java, RatingService.java
Description	All business logic errors across all three services use the same generic RuntimeException. The controller cannot distinguish between 'not found' (HTTP 404), 'duplicate' (HTTP 409), and 'validation failed' (HTTP 400) — all end up as 400. Any unexpected RuntimeException from the database or JPA layer could accidentally be caught and treated as a business error.

Code example	// SellerService.java
	throw new RuntimeException("User not found");
	throw new RuntimeException("Seller profile already exists");
	// RatingService.java
	throw new RuntimeException("Score must be between 1 and 5.");
	throw new RuntimeException("Invalid or already-used code.");

Proposed fix	Define custom exception classes: ResourceNotFoundException, DuplicateEntityException, ValidationException. Add a @ControllerAdvice to map them to the correct HTTP status codes (404, 409, 400). Proposed as follow-up issue due to scope.

Fix code	public class ResourceNotFoundException extends RuntimeException { ... }
	public class DuplicateEntityException extends RuntimeException { ... }
	public class ValidationException extends RuntimeException { ... }
	
	@ControllerAdvice
	public class GlobalExceptionHandler {
	  @ExceptionHandler(ResourceNotFoundException.class)
	  public ResponseEntity<?> handleNotFound(ResourceNotFoundException e) {
	  return ResponseEntity.status(404).body(Map.of("error", e.getMessage()));
	  }
	}

Summary table

#	Smell	Category	Severity	Files
1	Duplicated badge rendering logic	Duplicated code (DRY violation)	Medium	FarmListingsPage.jsx lines 68–86, SavedFarms.jsx lines 31–49
2	Calling Java getter methods on plain JSON objects	Inappropriate intimacy / bug	High (runtime error)	FarmListingsPage.jsx line 92, SavedFarms.jsx line 55
3	React component named with lowercase letter	Naming convention violation	Medium	SavedFarms.jsx line 8
4	Silent error handling (catch(console.error))	Improper error handling	Medium	FarmListingsPage.jsx line 29, SavedFarms.jsx line 21, FarmListing.jsx lines 53–56
5	getAllFarms duplicated in ratings.js	Duplicated code (DRY violation)	Low-medium	farm.js lines 5–8, ratings.js lines 6–9
6	getFarm(id) fetches all farms to find one	Inefficient algorithm / lazy class	Medium	farm.js lines 12–15
7	Long if-else chain for sorting (switch statement smell)	Switch statements	Low-medium	FarmListingsPage.jsx lines 54–62
8	RuntimeException used for all business logic errors	Inappropriate exception type	Medium	SellerService.java, BuyerService.java, RatingService.java

Commits for proposed fixes

Larger fixes (custom exceptions, DB aggregate for ratings, compare page) are proposed as follow-up issues due to scope.

Commit	Change	Smell fixed
fix: use correct JSON property access for farm images	Replace farm.getUser().getProfilePicture() with farm.user?.profilePicture	Smell #2
fix: rename savedFarms component to SavedFarms (PascalCase)	Rename function and export in SavedFarms.jsx	Smell #3
refactor: remove duplicate getAllFarms from ratings.js	Import from farm.js instead	Smell #5
refactor: replace sort if-else chain with lookup table	Add SORT_COMPARATORS map in FarmListingsPage.jsx	Smell #7

AI disclosure: All test cases in activity 2 and all code smell analyses in activity 3 are my

own work. The ideas, test scenarios, bug identifications, code review findings, and proposed fixes were developed by me through hands on testing and reading the codebase, though some stuff I was confused about I asked Claude to help clear up. I used Claude (by Anthropic) as a formatting and organizational tool to structure my notes into the final document layout, including generating the Word files with tables and consistent styling. Claude did not generate the test cases, identify the bugs, or perform the code review.